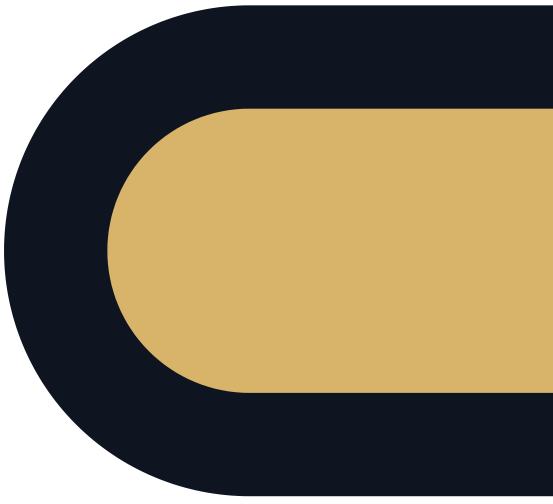
Solvra

TECHNOLOGY PLAN · V1

Solvra Technology Plan

From zero to scale: every technology, why it's there, and what changes as Solvra grows

حلولٌ بإتقان.



Version 1 · 27 September 2026 · Supersedes the stack notes in 01-product-discovery.md and 05-tech-stack.md.

1. How decisions are made

Seven principles decide every choice in this plan. When a new tool is proposed, it must pass these first.

1. **Spend follows revenue.** Infrastructure grows only when paying customers need it. The code is written for scale from day one; the servers are not.
2. **Boring, proven technology.** Every core component has been run in production by thousands of companies for years. Novelty lives in the product, not the stack.
3. **One language everywhere.** TypeScript in the browser, the API, the workers and (later) the mobile app. One person can move across the whole system.
4. **A modular monolith, not microservices.** One codebase and one database, divided into modules with strict boundaries. Any module can become its own service later, when there is a team and a reason.
5. **Correctness before speed.** Solvra handles money, deposits and legal documents. Transactions, constraints, append-only ledgers and audit logs come first.
6. **Portability.** Everything runs in Docker, on vanilla Postgres, with S3-compatible storage. If Iraq introduces data-localization rules, moving into an Iraqi data center is a configuration change, not a rewrite.
7. **Iraq-first.** Arabic and Kurdish right-to-left, dinars without decimals, weak mobile signal, WhatsApp, FIB/ZainCash/cash. These are requirements, not features.

2. The five stages at a glance

Stages are reached by **triggers** (evidence), not by dates.

STAGE	NAME	ENTER WHEN…	WHAT RUNS	MONTHLY COST (EST.)
0	Validation	Now	Nothing to host. Spreadsheets, shared folder, WhatsApp, a static "coming soon" page	**$0**
1	Foundation / MVP	Validation passed: ≥ 2 businesses paid for the manual service	One small server running everything	**≈ €6–8**
2	First paying customers	Real customers depend on it daily; ~10 organizations	Managed database, staging environment, paid messaging, payments live	**≈ $30–60**
3	Growth	~50–100 paying organizations, a second product, or a first hire	Several app servers, separate workers, cache, read replica	**≈ $150–400**
4	Scale	~1,000+ organizations, a team of 5–8+, enterprise customers or localization rules	Extracted services where justified, analytics warehouse, possibly an Iraqi data center	**$1,000+**

Costs exclude people, domains and per-message fees. They are estimates at September 2026 prices.

3. Stage 0: Validation ($0)

Goal: prove people pay before writing production code.

NEED	TOOL	COST
Customer records, shipment files, deposit schedules	Google Sheets / Excel + a shared Drive or OneDrive folder	Free
Talking to customers	WhatsApp Business app (free), with the Solvra avatar and a greeting message	Free
Intake forms	Google Forms or Tally	Free
"Coming soon" website	A static page on **Cloudflare Pages** (free, commercial use allowed), with the brand kit	Free
Brand, poster, mockups	Already done (`Visual Identities/Solvra`)	Done

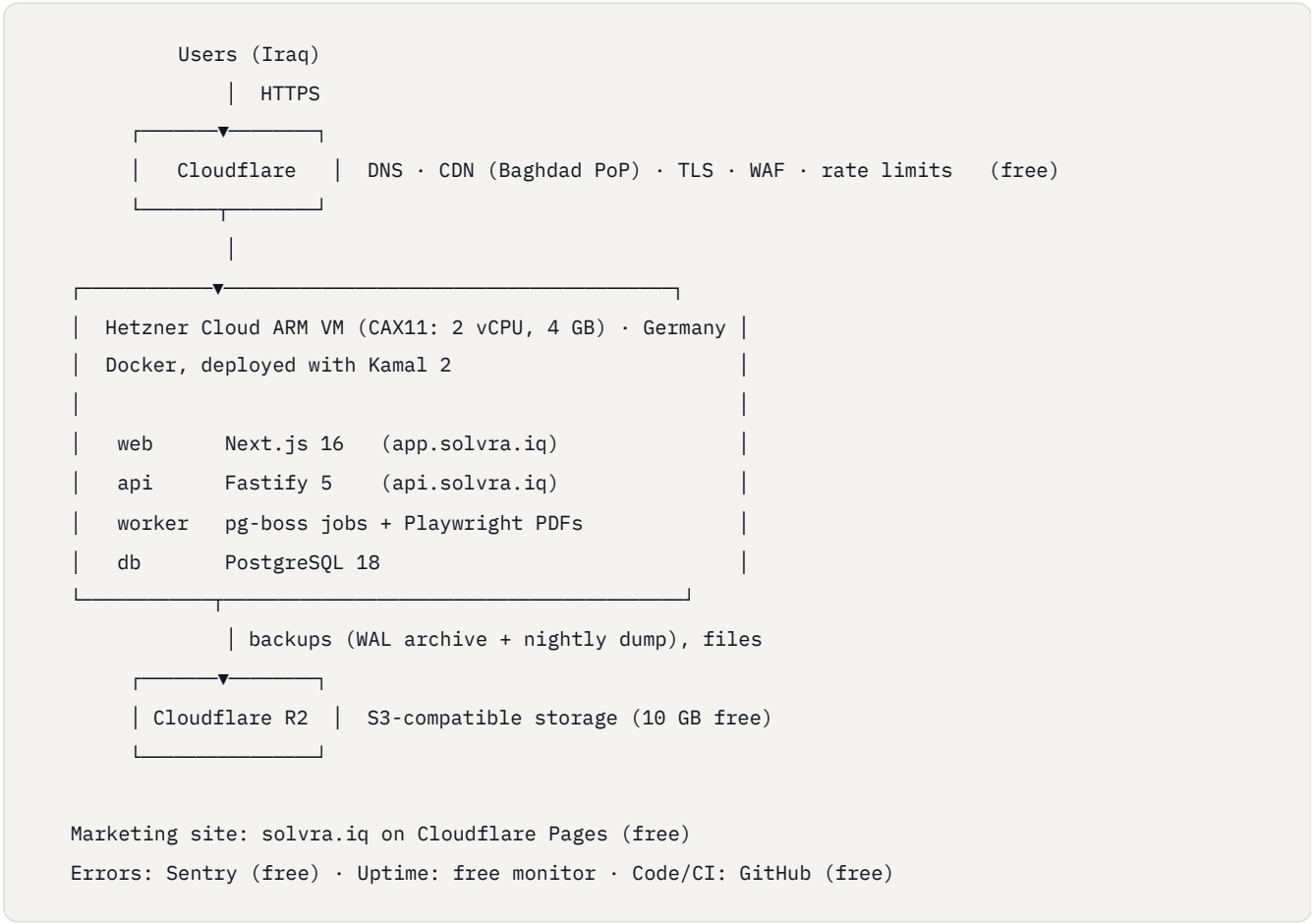
What you build in this stage: nothing that needs a server. You *may* create the repository skeleton (Stage 1, step 1) so that you're ready, but you don't deploy.

Exit trigger: the go/no-go memo from `04-final-decision.md` says go: at least 2 businesses paid and at least 1 accountant wants it for more clients.

4. Stage 1: Foundation / MVP (≈ €6–8 per month)

Goal: replace the manual service with software for the first few pilot customers.

Architecture



Setup order

1. **Monorepo skeleton:** pnpm, Turborepo, TypeScript strict, Biome, Vitest, GitHub Actions.
2. **`packages/core`:** money, IDs, time, errors, Arabic normalization, with tests.
3. **`packages/db`:** Drizzle schema, migrations, row-level security, a tenant-transaction wrapper, with tests against a real Postgres (Testcontainers).

4. **`packages/ui`:** React Aria + Tailwind + the brand tokens, with right-to-left layout and dark mode from the first component.
5. **Auth** in `apps/api` (Better Auth: organizations, email magic links), with the session shared with `apps/web`.
6. **Infrastructure:** one VM, Kamal deploy, Cloudflare DNS, backups to R2, Sentry, an uptime monitor.
7. **The first Shipments screens:** only the workflow the pilot customers actually paid for.

What's deliberately missing

- A managed database: Postgres runs on the VM, with continuous backups to R2.
- Redis/Valkey: pg-boss uses Postgres.
- Paid messaging: use free WhatsApp click-to-chat links (`wa.me`) and email login links.
- Staging: one production environment plus local development. Preview data comes from seed scripts.
- Analytics tools.

Risk and its mitigation

A single VM is a single point of failure. That's acceptable for pilot customers **only if** backups are continuous, stored off-server (R2) and **restored in a test at least monthly**. Keep a written runbook: "server is dead → new VM → restore → DNS" should take under an hour.

Exit trigger: pilots use it daily, around 10 organizations pay, the database passes ~1 GB, or a customer asks "what happens if your server dies?"

5. Stage 2: First paying customers (≈ $30–60 per month)

Goal: professional reliability, paid for by revenue.

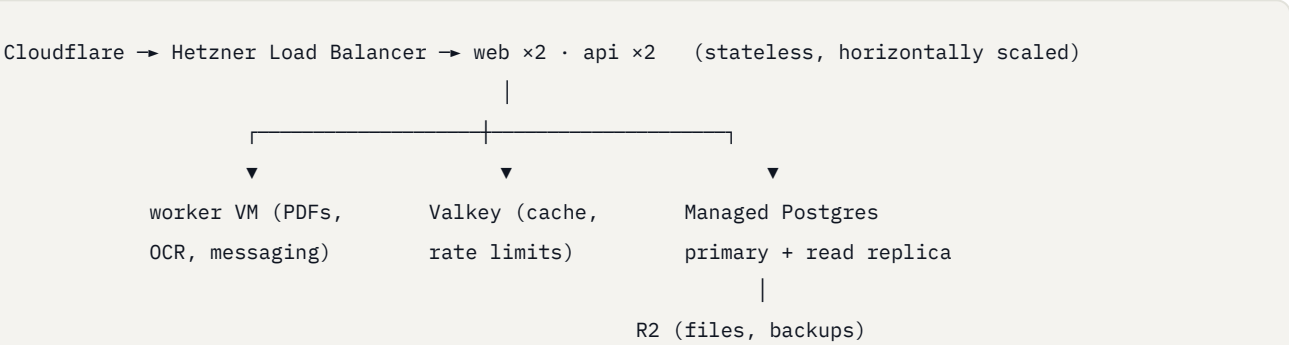
CHANGE	WHY
Managed PostgreSQL 18 in Frankfurt (e.g., Neon paid plan or Crunchy Bridge)	Point-in-time recovery, automated backups and failover without doing database operations yourself. The app VM stays on Hetzner; latency between them is a few milliseconds.
Staging environment: a second small VM or a database branch	Test migrations on a copy of real-shaped data before production
Phone login (OTP) through an Iraqi SMS/WhatsApp gateway	How Iraqi users expect to sign in; costs cents per code
WhatsApp Cloud API utility templates	Automatic "document missing" and "deposit recorded" messages. Priced per message.
Payments live: FIB API + webhooks, ZainCash, manual cash/transfer invoices	Customers pay Solvra digitally; billing and entitlements come from `packages/billing`

CHANGE	WHY
Product analytics: PostHog (free tier)	See which features are used; cut the rest
Status page + incident notes	Trust with paying customers

Exit trigger: ~50–100 paying organizations, response times (p95) above ~500 ms under normal load, database CPU consistently above ~60%, a second product, or a first hire.

6. Stage 3: Growth (≈ $150–400 per month)

Goal: handle more customers and more products without re-architecting.



CHANGE	TRIGGER / REASON
2+ app servers behind a load balancer	Load and zero-downtime deploys. The apps are stateless, so this is configuration.
Separate worker server	PDF rendering (Chromium) and imports shouldn't slow the API
Valkey (open-source Redis fork)	Caching hot reads, rate limiting, short-lived sessions
Read replica	Reports and dashboards move off the primary
Feature flags + per-organization entitlements	Roll out safely; sell plans per product
OpenTelemetry → Grafana Cloud	Traces and metrics across web, API and workers
Search engine (Meilisearch/Typesense), *only if* Postgres search is too slow	Large Arabic text search
Second product module (e.g., Solvra Service)	Added as a package; reuses auth, billing, files, notifications and audit

Exit trigger: ~1,000+ organizations, a team of 5–8+ engineers, enterprise customers with SLAs, or a data-localization law.

7. Stage 4: Scale ($1,000+ per month)

Goal: a multi-product company with a team.

- **Extract services only where justified.** A module with a very different load profile or a dedicated team becomes its own service. The module boundaries and outbox events already exist, so extraction is mechanical.
- **Job queues:** move hot queues from pg-boss to BullMQ + Valkey if throughput demands it. The outbox pattern stays.
- **Partition large tables** (audit, events, messages) by month.
- **Analytics warehouse** (e.g., ClickHouse) fed from the outbox, for reporting across products.
- **Enterprise features:** SSO/SAML, audit exports, a public API with webhooks.
- **Mobile:** an Expo (React Native) app reusing `core`, `api-client` and the Zod schemas, if the PWA isn't enough.
- **Offline sync at scale** for field products: evaluate PowerSync or ElectricSQL (Postgres sync engines).
- **Data localization:** run the same Docker services in an Iraqi data center, with Postgres self-managed or with a local provider, and MinIO instead of R2.
- **Kubernetes:** only if the team has operations capacity. Managed containers or Kamal on more servers may remain enough.

8. Every technology: what, why, and what changes

Each entry covers **what** it is, **why** Solvra uses it, **instead of** what, the **stage** it's introduced, what changes **later**, and one key **best practice**.

8.1 Foundations

TypeScript (strict mode)

- **What:** JavaScript with static types.
- **Why:** one language across web, API, workers and mobile. Types catch money and tenant bugs before users do.
- **Instead of:** plain JavaScript (too loose for financial code); Go or Python backends (a second language for one person).
- **Stage:** 1 · **Later:** unchanged.
- **Best practice:** `strict: true`, no `any` in domain code, validate every external input with Zod at the boundary.

Node.js 24 LTS

- **What:** the JavaScript runtime for the API and workers.
- **Why:** Active LTS, supported until May 2028. Mature, fast, huge ecosystem.
- **Instead of:** Bun or Deno (promising, but a less proven long-term production record).
- **Stage:** 1 · **Later:** upgrade to the next LTS (26) during its Active LTS window.
- **Best practice:** pin the version in `.nvmrc`, `package.json` engines and Docker images; upgrade deliberately, never implicitly.

pnpm + Turborepo (monorepo)

- **What:** one repository holding every app and package; pnpm manages dependencies, and Turborepo caches builds and tests.
- **Why:** shared code (money, auth, UI) without publishing packages. Atomic changes across API and web.
- **Instead of:** separate repositories (version drift); Nx (heavier).
- **Stage:** 1 · **Later:** remote build cache when CI slows down.
- **Best practice:** enforce module boundaries: products import the platform core, never each other.

Biome

- **What:** a single fast tool for linting and formatting.
- **Why:** replaces ESLint + Prettier with one config and much faster runs.
- **Stage:** 1 · **Later:** unchanged.
- **Best practice:** run it in CI and in a pre-commit hook; formatting is never discussed in review.

8.2 Frontend

Next.js 16 (App Router)

- **What:** the React framework for `app.solvra.iq` (the product) and `solvra.iq` (the marketing site).
- **Why:** server rendering gives fast first loads on weak Iraqi mobile networks. You already know it. Turbopack is stable and the default.
- **Instead of:** a plain React single-page app (slow first load); Remix/React Router (fine, but less familiar to you).
- **Stage:** 1 (marketing site as a static export in Stage 0) · **Later:** unchanged; product UIs are added as route groups.
- **Best practice:** keep business logic **out** of Next.js. Pages call the API; they never talk to the database directly.

Tailwind CSS v4 + Solvra tokens

- **What:** utility-first CSS driven by the brand's design tokens (`tokens.css`).
- **Why:** consistent styling at speed. Logical properties (`ps-*` , `me-*`) make right-to-left automatic.
- **Stage:** 1 · **Later:** unchanged; each product sets only `--color-product` .
- **Best practice:** never write `left` / `right` ; always use start/end. Never hard-code colours; use tokens.

React Aria Components → `@solvra/ui`

- **What:** accessible, unstyled components from Adobe, styled with Tailwind into Solvra's design system.
- **Why:** the best right-to-left, locale and accessibility support available. Arabic date pickers, number inputs and calendars are where other libraries fail.
- **Instead of:** shadcn/ui on Radix (good, but weaker i18n for dates and numbers); MUI (heavy, hard to brand).
- **Stage:** 1 · **Later:** grows into the full Solvra design system, shared by every product.
- **Best practice:** build components once in `@solvra/ui` and test them in Arabic first.

next-intl (ICU messages)

- **What:** translations and formatting for Arabic (`ar`), Kurdish Sorani (`ckb`) and English (`en`).
- **Why:** ICU handles Arabic plural and gender rules properly.
- **Stage:** 1 (Arabic + English); Kurdish when you sell in the Kurdistan Region.
- **Best practice:** no hard-coded strings. Write Arabic copy natively, not as a translation. Use Latin digits by default, configurable per organization.

TanStack Query + TanStack Table

- **What:** server-state caching and data tables.
- **Why:** shipments, ledgers and document lists are table-heavy; caching makes the app feel fast on slow networks.
- **Stage:** 1 · **Later:** unchanged.
- **Best practice:** the server is the source of truth; invalidate queries after mutations instead of hand-updating state.

React Hook Form + Zod

- **What:** forms, validated by the same Zod schemas the API uses.
- **Why:** one definition of "valid" in the browser and on the server.
- **Stage:** 1 · **Later:** unchanged.
- **Best practice:** the schemas live in a shared package; the API always re-validates, and the browser only helps the user.

Serwist + Dexie (PWA and offline)

- **What:** Serwist builds the service worker (installable app, offline cache); Dexie wraps IndexedDB for local data and an outbox of pending actions.
- **Why:** Iraqi connectivity is uneven. Photos, documents and forms must never be lost.
- **Stage:** 1 (installable + offline shell), fuller offline for field products in Stage 3.
- **Later:** a sync engine (PowerSync/ElectricSQL) for field products in Stage 4.
- **Best practice:** every offline action carries a client-generated ID and an idempotency key, so retries can never duplicate records.

8.3 Backend

Fastify 5 (separate API app)

- **What:** a fast, mature Node.js web framework for `api.solvra.iq`.
- **Why:** business logic lives in one place that the web app, offline sync, a future mobile app, partners and payment webhooks all share.
- **Instead of:** Next.js server actions only (couples logic to the UI); Express (older, slower, weaker typing); NestJS (heavy for a solo developer).
- **Stage:** 1 · **Later:** scales horizontally behind a load balancer (Stage 3). Modules can be extracted into services (Stage 4).
- **Best practice:** versioned routes (`/v1`), additive changes only, and an OpenAPI spec as the contract.

Zod + OpenAPI (contracts)

- **What:** Zod schemas define every request and response, and an OpenAPI spec is generated from them. A typed client (`packages/api-client`) is generated from the spec.
- **Why:** type safety end to end, without GraphQL, and a standard format for future partners.
- **Stage:** 1 · **Later:** published as a public API for partners (Stage 4).
- **Best practice:** never hand-write API types in the frontend; regenerate the client in CI.

Domain modules (`packages/shipments` , `billing` , ...)

- **What:** framework-free TypeScript packages holding each product's business rules.
- **Why:** the core of the modular monolith. Rules are testable without HTTP or a UI, and extractable later.
- **Stage:** 1 · **Later:** one package per product, and some become services at Stage 4.
- **Best practice:** products depend on the platform core, never on each other. Cross-product needs go through events.

Better Auth (self-hosted)

- **What:** authentication and organizations as a TypeScript library, stored in your own Postgres.
- **Why:** users, organizations and roles live in your database, not in a foreign SaaS. It supports phone-number OTP and multi-tenant organizations, and one Solvra account works across all products (cookie on `.solvra.iq`).
- **Instead of:** Clerk/Auth0 (cost per user, data abroad, lock-in); building auth yourself (security risk).
- **Stage:** 1 (email magic links), 2 (phone OTP), 4 (SSO/SAML for enterprise).
- **Best practice:** short sessions for staff roles, two-factor authentication for owners and admins, rate-limited OTP endpoints.

pg-boss + transactional outbox (background work)

- **What:** pg-boss is a job queue stored in Postgres. The outbox is a table where every domain event is written in the same transaction as the business change.
- **Why:** no extra infrastructure, and jobs can't be lost or created for changes that rolled back.
- **Instead of:** BullMQ + Redis on day one (another service to run); cloud queues (lock-in).
- **Stage:** 1 · **Later:** hot queues may move to BullMQ + Valkey (Stage 4). The outbox stays forever.
- **Best practice:** every job is idempotent and safe to run twice.

Playwright (PDF rendering)

- **What:** headless Chromium turning HTML templates into PDFs: statements, deposit schedules, invoices.
- **Why:** correct Arabic shaping and right-to-left layout. PDF libraries like React-PDF handle Arabic poorly. The brand kit already uses this method.
- **Stage:** 1 (in the worker) · **Later:** its own worker server (Stage 3).
- **Best practice:** render in the background and store the result in R2; never render inside a user's request.

8.4 Data

PostgreSQL 18

- **What:** the relational database.
- **Why:** ledgers, deposits and reconciliation need transactions, constraints and joins. Version 18 adds native `uuidv7()` and faster async I/O. It is the most portable serious database there is.
- **Instead of:** MongoDB (you'd have to enforce financial integrity in application code); MySQL (weaker features for this workload).
- **Stage:** 1 (on the VM), 2 (managed, Frankfurt), 3 (+ read replica), 4 (partitioning, possibly an Iraqi host).
- **Best practice:** use vanilla features only (no vendor extensions), so any provider or data center can host it.

Drizzle ORM

- **What:** a lightweight TypeScript ORM that stays close to SQL. Migrations are plain SQL files.
- **Why:** full control over SQL, row-level security and transactions, with type safety.
- **Instead of:** Prisma 7 (heavier; its abstraction fights row-level security and advanced SQL).
- **Stage:** 1 · **Later:** unchanged. Pin the version; 1.0 is in beta as of September 2026.
- **Best practice:** migrations are reviewed like code, applied in CI to a test database, and never edited once applied.

Multi-tenancy with row-level security (RLS)

- **What:** every tenant table has `org_id`. Postgres policies allow each request to see only its own organization's rows.
- **Why:** isolation is enforced by the database even if application code has a bug. That's essential when businesses trust you with financial records.
- **Stage:** 1 · **Later:** unchanged. Very large customers can move to a dedicated schema or database at Stage 4.
- **Best practice:** every request runs inside a transaction that sets `app.org_id`; product code has no "global" query path.

Search: Postgres full-text + `pg_trgm`, with Arabic normalization

- **What:** built-in search, plus a normalization function applied to every searchable field.
- **Why:** no extra service. Normalization makes Arabic search work: strip diacritics, unify ه → ة, ي → ى, أ/إ/آ → ا, and convert Eastern Arabic digits to Latin.
- **Stage:** 1 · **Later:** Meilisearch or Typesense only if search outgrows Postgres (Stage 3+).
- **Best practice:** store the normalized text in a generated column with an index.

Cloudflare R2 (object storage)

- **What:** S3-compatible file storage for documents, photos, PDFs and backups.
- **Why:** 10 GB free, no egress fees, and the S3 API means you can move to MinIO in an Iraqi data center without code changes.
- **Stage:** 1 · **Later:** MinIO or a local S3 provider if data must stay in Iraq.
- **Best practice:** uploads go directly from the browser with presigned URLs, stored under per-organization prefixes, and downloads use short-lived signed links.

8.5 Iraq-specific integrations

Messaging: WhatsApp, SMS OTP, email (`packages/notify`)

- **What:** one interface for sending messages, with an adapter per channel: Meta WhatsApp Cloud API, an Iraqi OTP gateway (SMS/WhatsApp), and Resend or Amazon SES for email.
- **Why:** WhatsApp is how Iraqi businesses communicate. Behind one interface, providers can change without touching products.
- **Stage:** 1 (free `wa.me` links + email), 2 (paid OTP and WhatsApp templates).
- **Best practice:** every message is logged with delivery status. Never send marketing through utility templates.

Payments and billing (`packages/billing` , `packages/payments`)

- **What:** plans, entitlements (which organization has which product) and invoices, built in-house. Payments go through provider adapters: FIB (REST API + webhooks), ZainCash, Qi via a local gateway, and manual cash or transfer.
- **Why:** Stripe doesn't serve Iraq, and cash and transfers are normal here. Owning billing lets you price per organization and per product.
- **Stage:** 1 (manual invoices recorded in the system), 2 (FIB/ZainCash live).
- **Best practice:** payment webhooks are verified, idempotent and reconciled daily against provider statements.

AI (optional, `packages/ai`)

- **What:** the Claude API behind an adapter, for example to extract fields from supplier invoices and certificates of origin.
- **Why:** it saves real typing on document-heavy workflows, but it's never required for the product to work.
- **Stage:** 3, when a validated workflow needs it.
- **Best practice:** a human confirms extracted data before it touches money. Never send customer documents to a provider without saying so in your terms.

8.6 Infrastructure and operations

Hetzner Cloud (Germany)

- **What:** virtual servers in Nuremberg/Falkenstein.
- **Why:** very low cost, EU stability, and about 70–90 ms from Baghdad. Gulf regions are not used because AWS data centers in the UAE and Bahrain were physically damaged in March 2026.
- **Stage:** 1 (one ARM CAX11), 3 (several servers + a load balancer).
- **Later:** an Iraqi data center if the law requires it; the same containers run there.
- **Best practice:** SSH keys only, firewall closed except 80/443 via Cloudflare, automatic security updates.

Docker + Kamal 2

- **What:** every service runs in a container. Kamal deploys containers to plain servers with zero downtime, configured in one file.
- **Why:** reproducible, portable and simple. No Kubernetes to learn or run.
- **Stage:** 1 · **Later:** Kamal across more servers (Stage 3); Kubernetes only if a team with operations capacity wants it (Stage 4).
- **Best practice:** images are built in CI, tagged with the commit, and deployed the same way every time. Never edit a server by hand.

Cloudflare (DNS, CDN, TLS, WAF, Pages)

- **What:** the network edge in front of everything, plus free static hosting for the marketing site.
- **Why:** it has a Baghdad point of presence (fast static assets inside Iraq), free TLS, DDoS protection and rate limiting.
- **Stage:** 0 (Pages) and 1 onward · **Later:** unchanged.
- **Best practice:** the origin server accepts traffic only from Cloudflare.

Backups (WAL-G or pgBackRest → R2)

- **What:** continuous archiving of Postgres changes, plus nightly full dumps, to R2.
- **Why:** on a single server, backups are the entire safety net.
- **Stage:** 1 (self-managed), 2+ (the managed provider's point-in-time recovery, plus your own logical dump to R2 as a second copy).
- **Best practice: restore-test monthly**, and keep a one-page recovery runbook.

Observability: Sentry, pino, OpenTelemetry, uptime

- **What:** Sentry for errors (free plan), structured JSON logs (pino), a free uptime monitor; OpenTelemetry traces and metrics to Grafana Cloud later.
- **Why:** you can't fix what you can't see, especially when you're the only engineer.
- **Stage:** 1 (Sentry + logs + uptime), 3 (OpenTelemetry + Grafana).
- **Best practice:** never log personal data or document contents. Include `org_id` and a request ID in every log line.

CI/CD and testing: GitHub Actions, Vitest, Testcontainers, Playwright, Renovate

- **What:** on every push: typecheck, lint, unit tests (Vitest), database tests against real Postgres 18 (Testcontainers), end-to-end tests of key flows (Playwright), and a migration check. Renovate keeps dependencies current.
- **Why:** a solo developer needs the machine to catch regressions, especially in money and tenant isolation.
- **Stage:** 1 · **Later:** preview environments per pull request (Stage 3).
- **Best practice:** tests for money arithmetic, row-level security and idempotency are mandatory, not optional.

Product analytics: PostHog

- **What:** feature usage, funnels and session replays.
- **Why:** it shows what customers actually use, so you can cut features that don't earn their place.
- **Stage:** 2 (free tier) · **Later:** self-host or paid tier.
- **Best practice:** mask all document contents and personal data in replays.

9. Engineering best practices (non-negotiable from day one)

Data and money

1. **Money** is `bigint` in minor units plus an ISO currency code (IQD has 0 decimals, USD has 2). Never floats.
2. **Exchange rates** are stored per transaction with date and source (CBI official vs market).
3. **Ledgers are append-only.** Corrections are new entries, never edits.
4. **IDs** are `uuidv7()`. Human-readable references (e.g., `SHP-2026-0142`) come from per-organization counters.
5. **Time** is `timestamptz` in UTC and displayed in `Asia/Baghdad`. Business deadlines use the `date` type.
6. **Audit log** for every change to money or documents: who, what, when, before and after.

Architecture

7. **Tenant isolation** through row-level security on every tenant table.
8. **Idempotency keys** on every write endpoint and every job.
9. **Outbox events** for every state change another module cares about.
10. **Module boundaries:** products never import each other.
11. **Adapters** for everything provider-specific: storage, messaging, payments, AI.
12. **API contract:** `/v1`, OpenAPI, additive changes only.

Security

13. Secrets live in environment variables managed by Kamal, never in Git. Rotate them after any suspected leak.
14. Two-factor authentication for every admin account: GitHub, Cloudflare, Hetzner, the database provider, email.
15. Least privilege: the app's database user can't drop tables; a separate migration user runs migrations.
16. Rate limits on login, OTP and public endpoints (Cloudflare, plus limits in the app).
17. Dependencies are updated weekly (Renovate); security patches are applied within days.
18. Personal data is collected only when needed and never written to logs.

Operations

19. Every deploy is automated and reversible; roll back by redeploying the previous image.
20. Backups are restore-tested monthly.
21. Every incident gets a short written note: what happened, impact, fix, and how to prevent it.
22. Cost is reviewed monthly; any service over its stage budget needs a reason.

10. Deliberately not used (and why)

NOT USED	WHY
MongoDB for financial data	Financial integrity would depend on application code instead of the database
Microservices at the start	Operational cost with no benefit for one developer
Kubernetes before a team exists	A full-time job to run well
Firebase / Supabase for core data	Lock-in, and data held in foreign services
Clerk / Auth0	Per-user cost, identities stored abroad
GraphQL	OpenAPI REST is simpler for a small team and for partners
Edge / serverless functions for database logic	Containers next to the database are faster and cheaper
Gulf cloud regions	Physical war damage to AWS UAE/Bahrain data centers in 2026
"Always free" hobby tiers for customer data	Accounts can be suspended; some forbid commercial use

11. Risks and mitigations

RISK	MITIGATION
Single server failure (Stage 1)	Continuous backups to R2, monthly restore tests, a one-hour recovery runbook
Price increases (Hetzner raised prices twice in 2026)	Containers are portable; any VPS provider works
A provider shuts down or changes terms	Adapters for storage, messaging, payments and AI; vanilla Postgres
Iraqi data-localization law	The same containers run in an Iraqi data center; R2 → MinIO
Key-person risk (solo founder)	Everything as code (Kamal, migrations, CI), written runbooks, a password manager with an emergency-access contact
Surprise messaging costs	Free channels first; per-organization message quotas; monthly cost review
Security incident	Row-level security, two-factor authentication everywhere, least privilege, audit logs, incident notes

12. Day-one checklist (when Stage 1 starts)

- [ ] GitHub organization with two-factor authentication; private monorepo; branch protection on `main`
- [ ] pnpm + Turborepo + TypeScript strict + Biome + Vitest; CI running on every push
- [ ] `packages/core`: money, ids, time, Arabic normalization, with tests
- [ ] `packages/db`: Drizzle, first migration, row-level security, tenant transaction helper, Testcontainers tests
- [ ] `packages/ui`: tokens, fonts, right-to-left, dark mode, the first five components
- [ ] Better Auth: organizations, email magic links, sessions shared across subdomains
- [ ] Hetzner CAX11 + Kamal 2 + Cloudflare DNS (`app.`, `api.`), origin locked to Cloudflare
- [ ] Postgres 18 container + WAL archiving and nightly dumps to R2 + **one successful restore test**
- [ ] Sentry, structured logs, uptime monitor
- [ ] Marketing site on Cloudflare Pages with the brand kit
- [ ] A password manager holding every credential, with an emergency-access contact

Sources

- Next.js 16: https://nextjs.org/blog/next-16
- Node.js release schedule: https://endoflife.date/nodejs
- PostgreSQL 18 release notes: https://www.postgresql.org/docs/release/18.0/
- Drizzle vs Prisma, 2026: https://makerkit.dev/blog/tutorials/drizzle-vs-prisma
- Better Auth, organizations and phone number: https://better-auth.com/docs/plugins/organization · https://better-auth.com/docs/plugins/phone-number
- Hetzner 2026 price changes: https://northflank.com/blog/hetzner-cloud-server-price-increases
- Hetzner locations: https://docs.hetzner.com/cloud/general/locations/
- Neon free plan limits: https://neon.com/faqs/free-plan-limits-and-quotas
- Cloudflare R2 free tier: https://freetier.co/directory/products/cloudflare-r2
- Sentry free plan: https://costbench.com/software/developer-tools/sentry/free-plan/
- AWS Middle East data center strikes (2026): https://www.thestack.technology/get-your-data-out-now-says-aws-on-me-central-1/ · https://www.infoq.com/news/2026/03/aws-multiaz-conflict-outage/
- Iraq data-center plan (September 2026): https://www.iraqinews.com/iraq/iraq-communications-ministry-plans-data-centers-baghdad-basra-mosul-2026/
- FIB payments API: https://fib.iq/integrations/web-payments/